

Lecture 19

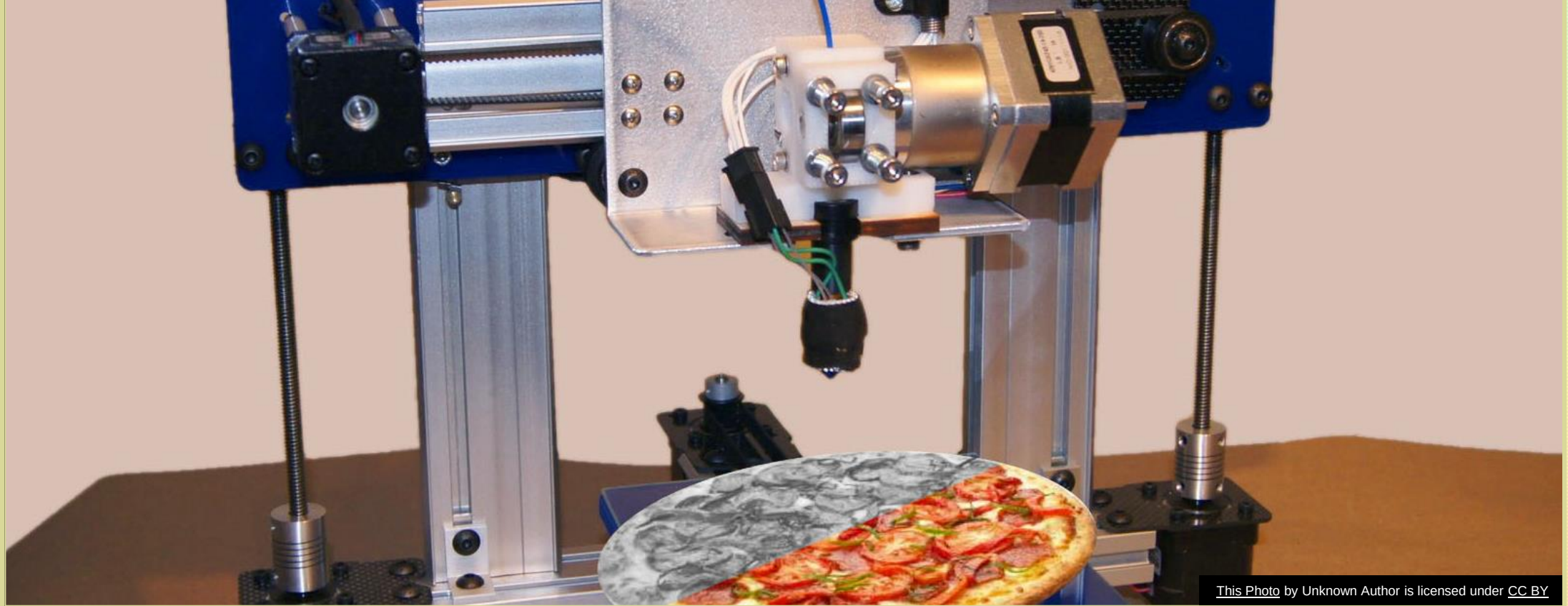
CS-317 Operating Systems



Book 1 Chapter 11

I/O Management





This Photo by Unknown Author is licensed under [CC BY](#)

Operating System Design Issues

Relevant to input and output devices





Design objectives

- Two objectives are paramount in designing the I/O facility:
 - Efficiency, and
 - Generality.



Efficiency

- Efficiency is important because I/O operations often form a bottleneck in a computing system.
- Multiprogramming allows some processes to be waiting on I/O operations while another process is executing. However, even with the vast size of main memory in today's machines, it will still often be the case that I/O is not keeping up with the activities of the processor. means: i/o operations are slower than cpu operations
- **Swapping** is used to bring in additional ready processes to keep the processor busy, but this is an I/O operation itself.
- The area that has received the most attention for improvement in efficiency is **disk I/O**.

Disk I/O (Disk Input/Output) means any operation that reads data from a disk or writes data to it, we improve it because disk i/o is slowest of all



Generality

- For simplicity and freedom from error, it is desirable to handle all devices in a uniform manner. This applies both to the way in which processes view I/O devices and to the way in which the OS manages I/O devices and operations.
- We use a hierarchical, modular approach to the design of the I/O function.
- This approach hides most of the details of device I/O in lower-level routines so that user processes and upper levels of the OS see devices in terms of general functions, such as read, write, open, close, lock, and unlock.



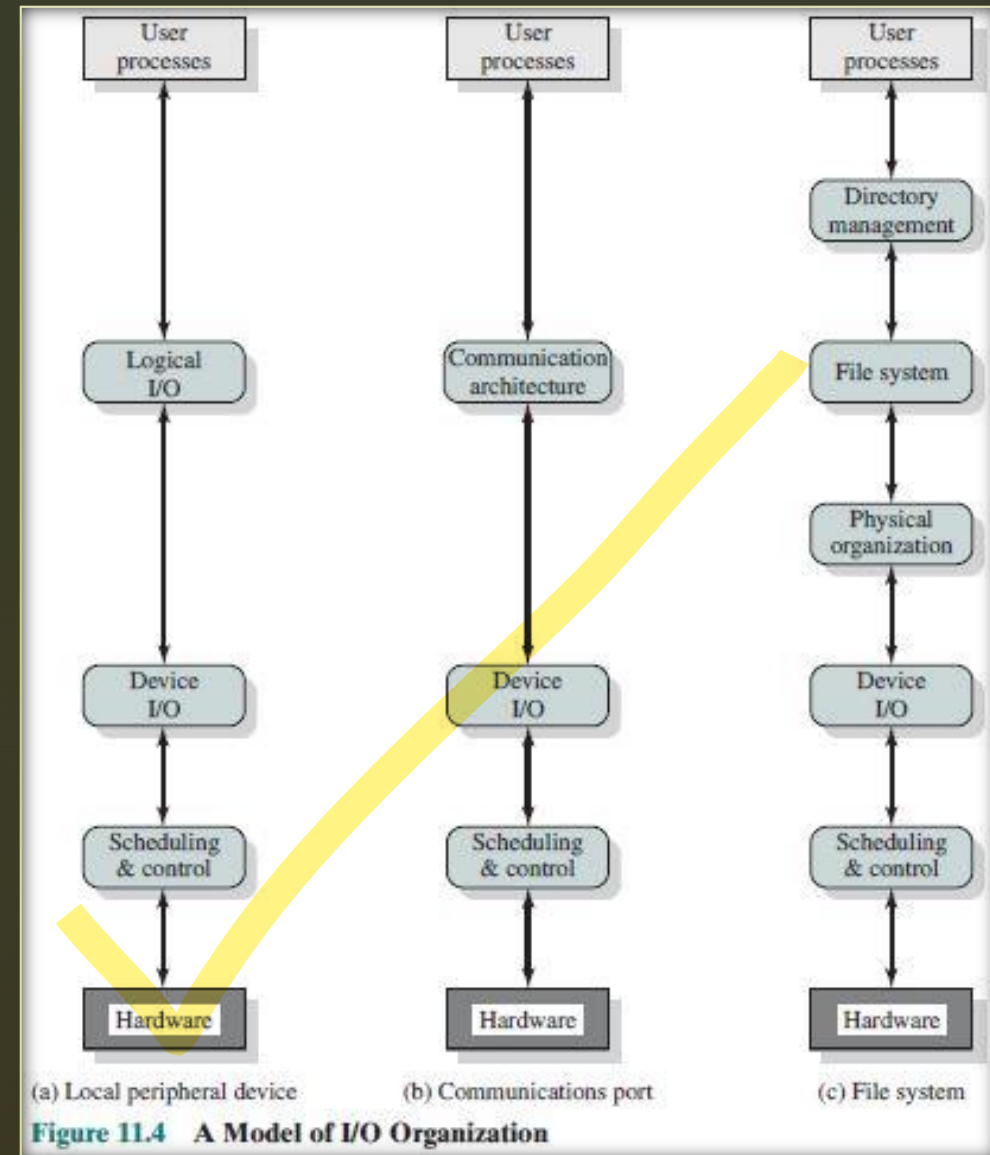
Layered organization of the OS

- The hierarchical philosophy is that the functions of the OS should be separated according to their complexity, their characteristic time scale, and their level of abstraction. Following this approach leads to an organization of the OS into a series of layers.
- Each layer performs a related subset of the functions required of the OS. It relies on the next lower layer to perform more primitive functions and to conceal the details of those functions. It provides services to the next higher layer.
- Ideally, the layers should be defined so that changes in one layer do not require changes in other layers.



Logical structure of the I/O Function

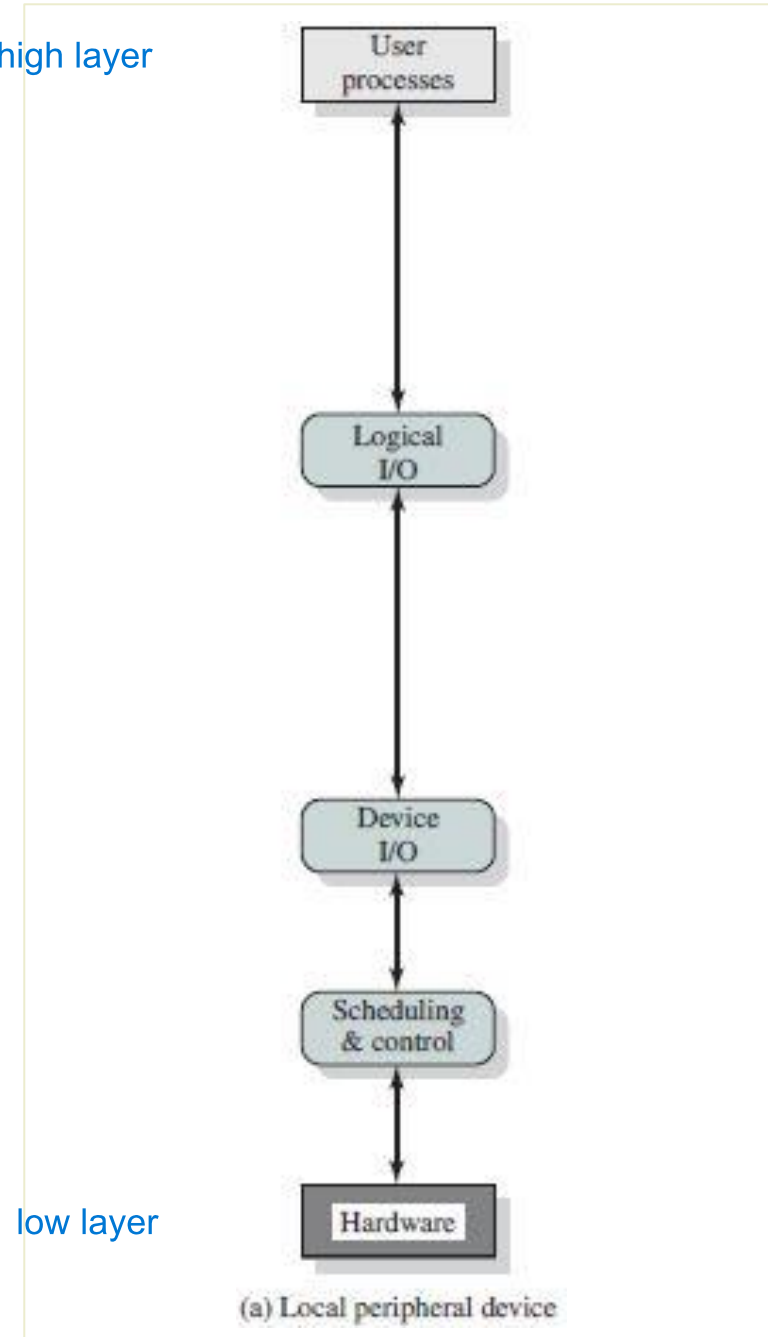
- Applying the hierarchical philosophy specifically to the I/O facility leads to the organization shown here.
- Most operating systems approach I/O in approximately this way.



Local peripheral device

- Logical I/O: The logical I/O module is concerned with **managing general I/O functions on behalf of user processes**, allowing them to deal with the device in terms of a **device identifier** and **simple commands such as open, close, read, and write**.

high layer



(a) Local peripheral device



Local peripheral device

- Device I/O: The requested operations and data (buffered characters, records, etc.) are converted into appropriate sequences of I/O instructions, channel commands, and controller orders. Buffering techniques may be used to improve utilization.

User
processes

Logical
I/O

Device
I/O

Scheduling
& control

Hardware

(a) Local peripheral device



Local peripheral device

- Scheduling and control: The actual queueing and scheduling of I/O operations occurs at this layer, as well as the control of the operations. Thus, interrupts are handled at this layer and I/O status is collected and reported. This is the layer of software that interacts with the I/O module and hence the device hardware.



(a) Local peripheral device



Communications port

- For a communications device, the I/O structure looks much the same as for a local peripheral device.
- The principal difference is that the logical I/O module is replaced by a communications architecture, which may itself consist of several layers. An example is TCP/IP.



(b) Communications port



File system

- Directory management: At this layer, symbolic file names are converted to identifiers that either reference the file directly or indirectly through a file descriptor or index table. This layer is also concerned with user operations that affect the directory of files, such as add, delete, and reorganize.



File system

- File system: This layer deals with the logical structure of files and with the operations that can be specified by users, such as open, close, read, and write. Access rights are also managed at this layer.



File system

- Physical organization: Logical references to files and records must be converted to physical secondary storage addresses, considering the physical track and sector structure of the secondary storage device. Allocation of secondary storage space and main storage buffers is generally treated at this layer as well.





This Photo by Unknown Author is licensed under [CC BY-SA-NC](#)

I/O Buffering

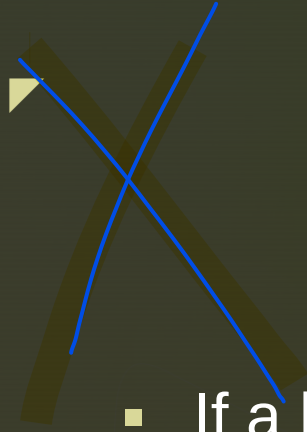
Details and types



Reading a block of data from disk

- A user process wishes to read a block of data from a disk one at a time having a length of 512 bytes. The data is to be placed within the address space of the process at location 1000 to 1511. We execute an I/O command (`Read_Block [1000, disk]`) to the disk unit and then wait.
- The waiting could either be:
 - Busy waiting (continuously testing the device status) , or,
 - The process will be blocked till the arrival of an interrupt.
- There are two problems with this approach. disk i/o is the slowest
 - The program is waiting for the relatively slow I/O to complete.
 - This approach interferes with swapping decisions by the OS. If paging is being used, at least the page containing the target locations must be locked into main memory.

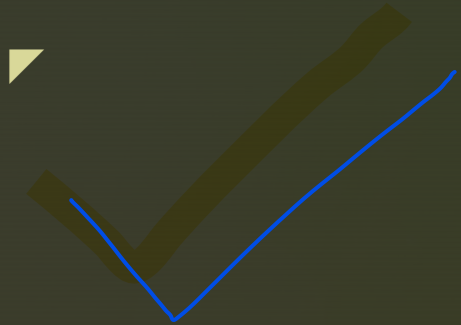




Buffering

- If a block is being transferred from a user process area to an I/O module, then the process is blocked during the transfer and the process may not be swapped out.
- To avoid these overheads and inefficiencies, it is sometimes convenient to **perform input transfers some time before the request is made** and to **perform output transfers some time after the request is made**. This technique is known as **buffering**.





Types of I/O devices

- A **block-oriented device** stores information in blocks that are usually of fixed size, and transfers are made one block at a time. Generally, it is possible to reference data by its block number. hard disk, ssd, usb flash drive
- A **stream-oriented device** transfers data in and out as a stream of bytes, with no block structure. keyboard, mouse, audi input devices



Pause the video!

Write your answer in a word file. Save it as <my_roll_no>_lecture_19. If your roll no is 500, the file will be named 500_lecture_19. Do not add <> signs to the filename.

Task# 1: Give at least two examples each for the two types of I/O devices.



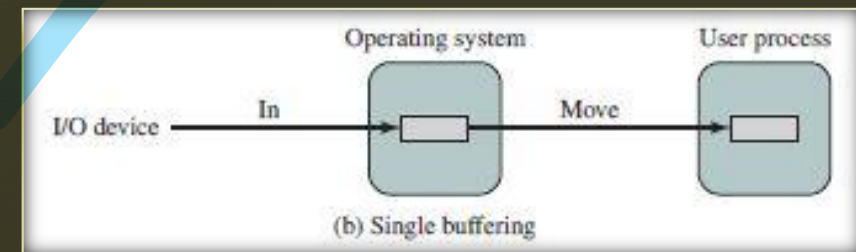
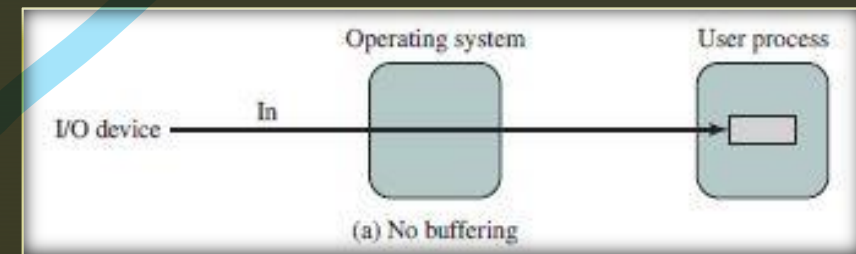
Single buffer

- When a user process issues an I/O request, the OS assigns a buffer in the system portion of main memory to the operation. Input transfers are made to the system buffer.
- For block-oriented devices, when the transfer is complete, the process moves the block into user space and immediately requests another block. This is called **reading ahead**, or **anticipated input**; it is done in the expectation that the block will eventually be needed.
- This is a reasonable assumption most of the time because data are usually accessed sequentially. Only at the end of a sequence of processing will a block be read in unnecessarily.



Single buffer

- This approach will generally provide a speedup compared to the lack of system buffering.
- The user process can be processing one block of data while the next block is being read in.
- The OS can swap the process out because the input operation is taking place in system memory rather than user process memory.



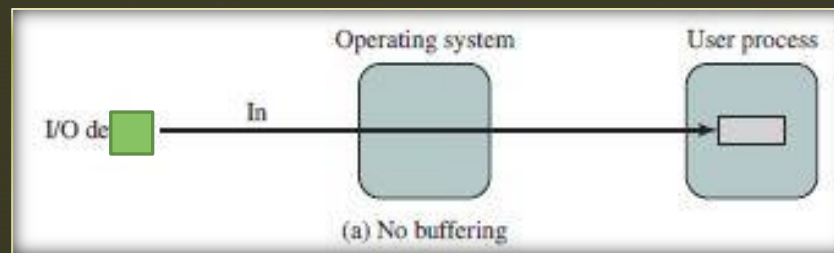
Single buffer

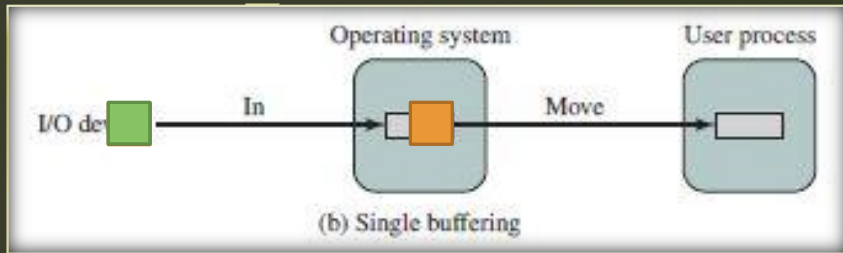
- The OS must keep track of the assignment of system buffers to user processes.
- If the I/O operation involves the same disk that is used for swapping, it hardly makes sense to queue disk writes to the same device for swapping the process out. This attempt to swap the process and release main memory will itself not begin until after the I/O operation finishes, at which time swapping the process to disk may no longer be appropriate.
- For block-oriented output, when data are being transmitted to a device, they are first copied from the user space into the system buffer, from which they will ultimately be written. The requesting process is now free to continue or to be swapped as necessary.



Performance comparison between single buffering and no buffering

- Time required to input one block = T
- Computation time between input requests = C
- Without buffering, execution time per block = $T + C$

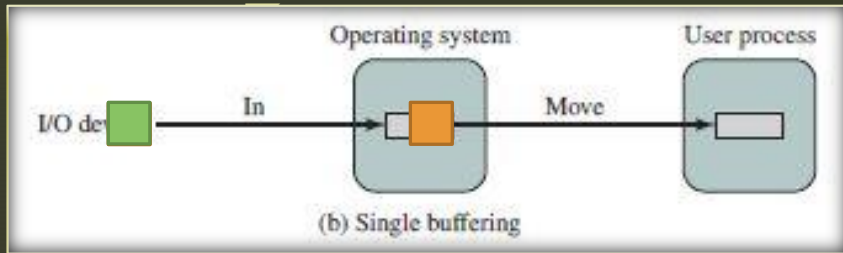




Performance comparison between single buffering and no buffering

- Time required to input one block $= T$
- With single buffer, assume time required to move the data from the system buffer to user memory $= M$
- Computation time between input requests $= C$
- Two things happen together: data transfer from the input device to the system buffer for the second block, and computation on the first block. So C and T overlap for consecutive blocks.
- If T is larger than C , we will complete computation on the first block, but the buffering of the second block will still be going on. So the total time that we will spend on a block will be $T + M$. because c will be covered in T





Performance comparison between single buffering and no buffering

- Time required to input one block $= T$
- With single buffer, assume time required to move the data from the system buffer to user memory $= M$
- Computation time between input requests $= C$
- If C is larger than T , we will complete buffering of the second block, but the computation on the first block will still be going on. So the total time that we will spend on a block is $C + M$.

After one block is read into memory:

CPU works on that data (calculations, processing, etc.)
This working time is called computation time



Performance comparison between single buffering and no buffering

- Time required to input one block = T
- Computation time between input requests = C
- Without buffering, execution time per block = $T + C$
- With single buffer, assume time required to move the data from the system buffer to user memory = M
- With single buffer, execution time per block = $\max [C, T] + M$.
- In most cases, execution time per block is substantially less with a single buffer compared to no buffer.



Single buffer and stream-oriented I/O

- For stream-oriented I/O, the single buffering scheme can be used in:
 - A line-at-a-time fashion, or
 - A byte-at-a-time fashion.



Line-at-a-time

- Line-at-a-time operation is appropriate for scroll-mode terminals (sometimes called dumb terminals).
- User input is one line at a time, with a carriage return signaling the end of a line, and output to the terminal is one line at a time.
- A line printer is another example of such a device.
- The buffer can be used to hold a single line.
- The user process is blocked during input, awaiting the arrival of the entire line.
- For output, the user process can place a line of output in the buffer and continue processing. It need not be blocked unless it has a second line of output to send before the buffer is emptied from the first output operation.



Byte-at-a-time

- Byte-at-a-time operation is used on forms-mode terminals, when each keystroke is significant, and for many other peripherals, such as sensors and controllers.
- In the case of byte-at-a-time I/O, the interaction between the OS and the user process follows the producer / consumer model.
- During input, the OS places a byte in the buffer from the device and the user process consumes it.
- During output, the user process places a byte in the buffer and the OS consumes it and passes it to the device.



Pause the video!

Write your answer in a word
file previously saved as
`<my_roll_no>_lecture_19`.

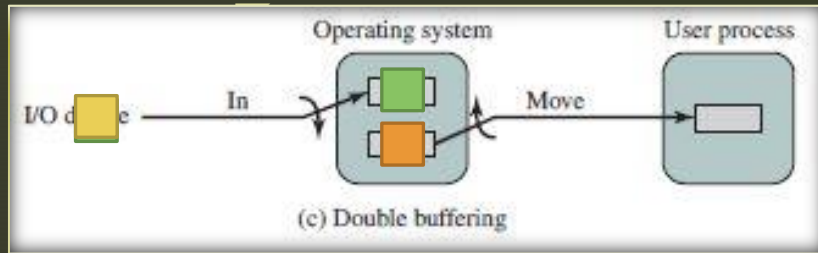
Task# 2: What is a forms-mode terminal?

> a terminal where input is taken in a structured format using predefined fields (forms) instead of free typing commands.

****Scroll mode terminal**** means:

> a terminal that displays output in a continuously scrolling screen where old lines move up as new lines are added.

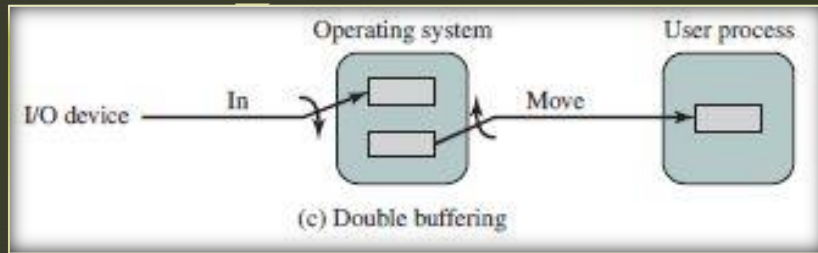




Double buffer

- An improvement over single buffering can be had by assigning two system buffers to the operation.
- A process now transfers data to (or from) one buffer while the operating system empties (or fills) the other.
- This technique is known as double buffering or buffer swapping.

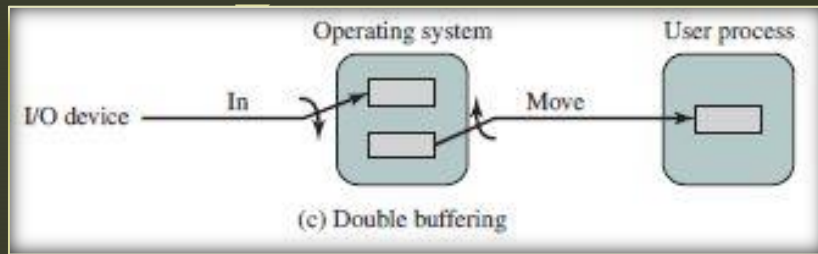




Double buffer and block-oriented I/O

- Let us assume $T = M$. This means the time to transfer a block from device to system buffer equals the time to transfer a block from system buffer to user space. So M gets overlapped with T for consecutive blocks.
- For block-oriented transfer, we can roughly estimate the execution time as $\max [C, T]$.
- If T is greater than C , the process completes computation of the first block, but the second block is still being transferred to the system buffer. So we must wait T seconds for the availability of the second block.

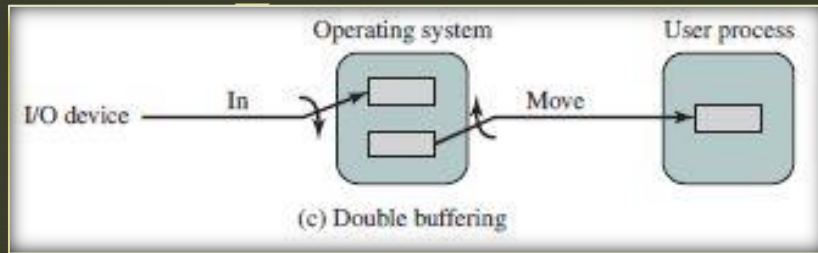




Double buffer and block-oriented I/O

- It is possible to keep the block-oriented device going at full speed if $T \geq C$. The device can transfer without delays between consecutive transfers, because the process computes in lesser time than the transfer time.
- If T is less than C , the second block arrives completely into the system buffer, but the process is still computing the first block. So the second block will be used by the process after C seconds.
- If $T < C$, double buffering ensures that the process will not have to wait for the I/O device.
since, the i/o is is working on fetchin the next block hence not dle
- In either case, an improvement over single buffering is achieved at the cost of increased complexity.

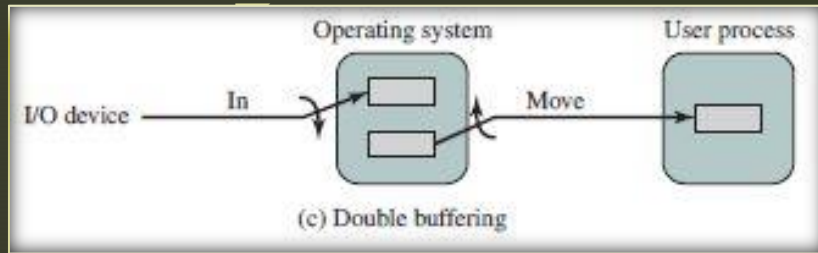




Double buffer and stream-oriented I/O

- Let us assume $T = M$. This means the time to transfer a block from device to system buffer equals the time to transfer a block from system buffer to user space. So M gets overlapped with T for consecutive blocks.
- For stream-oriented input, we have two alternative modes of operation.
- For line-at-a-time I/O, if $T > C$, the process finishes computation of the first line, but the second line is still being transferred. So the process gains access to the second line after T seconds, and blocks for $T - C$ seconds after computing the first line.

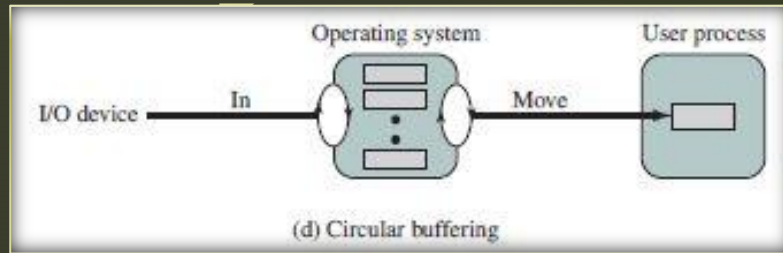




Double buffer and stream-oriented I/O

- For line-at-a-time I/O, if $T < C$, the second line is transferred earlier, but the process is still busy in computation of the first line. So the process accesses the second line after C seconds.
- The user process need not be blocked for input or output, unless $T > C$.
- For byte-at-a-time operation, the double buffer is equivalent to a single buffer of twice the length.
- In both cases, the producer / consumer model is followed.





Circular buffer

- A double-buffer scheme should smooth out the flow of data between an I/O device and a process.
- Double buffering may be inadequate if the process performs rapid bursts of I/O. In this case, the problem can often be solved by using more than two buffers.
- When more than two buffers are used, the collection of buffers is itself referred to as a circular buffer with each individual buffer being one unit in the circular buffer.
- This is simply the bounded-buffer producer / consumer model.



The utility of buffering

- Buffering is a technique that smooths out peaks in I/O demand.
- However, no amount of buffering will allow an I/O device to keep pace with a process indefinitely when the average demand of the process is greater than the I/O device can service, i.e., when $C \ll T$.
- Even with multiple buffers, all the buffers will eventually fill up and the process will have to wait after processing each chunk of data.
- However, in a multiprogramming environment, when there is a variety of I/O activity and a variety of process activity to service, buffering is one tool that can increase the efficiency of the OS and the performance of individual processes.





Disk Scheduling

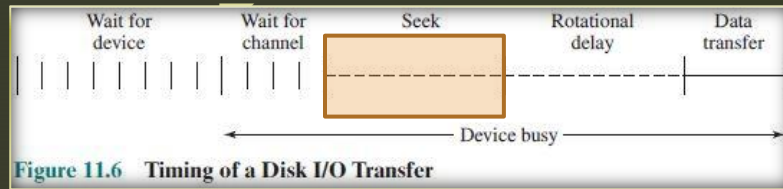
Concept and algorithms



Point of concern

- The speeds of processors and main memory have increased far more rapidly as compared to the speed of the disk.
- Disks are currently at least four orders of magnitude slower than main memory.
- The performance of disk storage subsystem is of vital concern, and much research has gone into schemes for improving it.

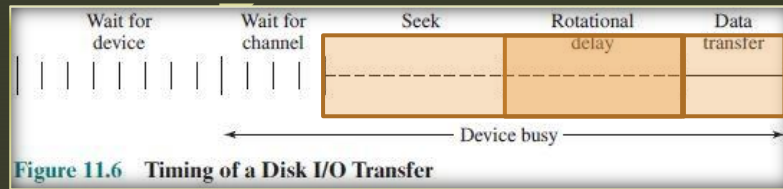




Disk performance parameters

- When the disk drive is operating, the disk is rotating at constant speed.
- To read or write, the head must be positioned at the desired track and at the beginning of the desired sector on that track.
- Track selection involves:
 - Moving the head in a movable-head system, or
 - Electronically selecting one head on a fixed-head system.
- On a movable-head system, the time it takes to position the head at the track is known as seek time.



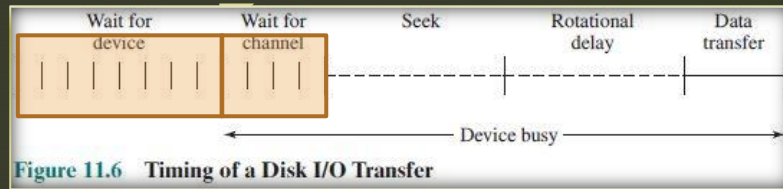


Disk performance parameters

- Once the track is selected, the disk controller waits until the appropriate sector rotates to line up with the head.
- The time it takes for the beginning of the sector to reach the head is known as rotational delay, or rotational latency.
- The sum of the seek time, if any, and the rotational delay equals the access time, which is the time it takes to get into position to read or write.
- Once the head is in position, the read or write operation is performed as the sector moves under the head; this is the data transfer portion of the operation; the time required for the transfer is the transfer time.

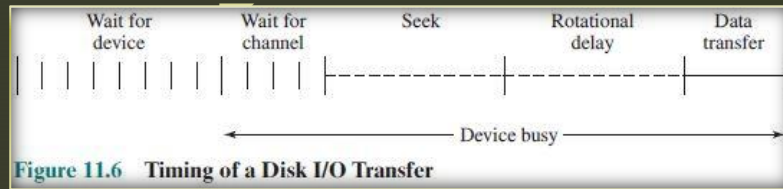


Additional delays



- In addition to the access time and transfer time, there are several queueing delays normally associated with a disk I/O operation.
- When a process issues an I/O request, it must first wait in a queue for the device to be available.
- If the device shares a single I/O channel or a set of I/O channels with other disk drives, then there may be an additional wait for the channel to be available.





Rotational Positional Sensing (RPS)

- When the seek command has been issued, the channel is released to handle other I/O operations.
- When the seek is completed, the device determines when the data will rotate under the head.
- As that sector approaches the head, the device tries to reestablish the communication path back to the host.
- If either the control unit or the channel is busy with another I/O, then the reconnection attempt fails and the device must rotate one whole revolution before it can attempt to reconnect, which is called an RPS miss.



Seek time

- Seek time is the time required to move the disk arm to the required track.
- It is difficult to estimate seek time.
- Seek time consists of two key components:
 - The initial startup time, and
 - The time taken to traverse the tracks that must be crossed to get to the desired track, called the traversal time.



dsk time can be improved by the redcing the ddiameter of th disk

Improving seek time

- Traversal time is not a linear function of the number of tracks but includes a settling time (time after positioning the head over the target track until track identification is confirmed).
- Much improvement comes from smaller and lighter disk components. The most common disk size today is 3.5 inches, reducing the distance that the arm must travel.
- A typical average seek time on contemporary hard disks is under 10 milliseconds.





$$T = \frac{1}{2r}$$

Rotational delay

- Rotational delay is the time required for the addressed area of the disk to rotate into a position where it is accessible by the read/write head.
- Disks rotate at speeds ranging from 3,600 rpm (for handheld devices such as digital cameras) up to 15,000 rpm.
- At 15,000 rpm, one full revolution takes 4 milliseconds.
Time = $(1/15000) \times 60 = 0.004$ seconds
- On average, the rotational delay will be 2 milliseconds.



Transfer time

- The transfer time to or from the disk depends on the rotation speed of the disk. We say that:

$$T = b/(rN)$$

where,

T = transfer time,

b = number of bytes to be transferred,

N = number of bytes on a track,

r = rotation speed, in revolutions per second.

- The total average access time, T_a , can be expressed as:

$$T_a = T_s + 1/(2r) + b/(rN)$$

where T_s is the average seek time. The second term represents average rotational delay.



What did we gather so far?

- We looked in detail at design issues involved in the I/O function of an OS.
- We discussed the logical structure of the I/O function.
- We understood the concept of buffering.
- We delved into different forms of buffering.
- We discussed disk scheduling.



Things to do

Read

Read the book.

Find

I will ask you to submit the word file previously saved as <my_roll_no>_lecture_19 in Google classroom.

Note

Write your questions on a piece of paper and keep it in front of you during the online session.

Email

If you have suggestions relevant to content or quality of this lecture, email them to me.

